

Autonomous Lane Following and RL-Based Obstacle Avoidance on a Duckiebot

Design, Implementation and Evaluation

Chitsidzo Varaidzo Nemazuwa and Kanaya Nisa Ozora

School of Computation, Information, and Technology, Technical University of Munich

March 24, 2026

Abstract — This report presents the design, implementation, and evaluation of an autonomous lane following and obstacle avoidance system deployed on a DB21J Duckiebot running the daffy software stack. The system combines a Finite State Machine (FSM) for high-level behavior switching with a Twin Delayed Deep Deterministic Policy Gradient (TD3) reinforcement learning agent for reactive obstacle avoidance. We describe the overall system architecture, the RL training and deployment pipeline, and the ROS packages developed or modified as part of this project.

1 Introduction to Duckietown

Duckietown is an open-source robotics education and research platform designed to study autonomous driving in a miniaturized urban environment. The platform consists of standardized road segments, traffic signs, and differential-drive robots called Duckiebots, which are equipped with a monocular camera, wheel encoders, and optionally a Time-of-Flight (ToF) distance sensor.

The software stack is built on ROS (Robot Operating System) and organized around a modular pipeline: raw camera images are processed by a line detector, projected onto the ground plane, and fed into a lane filter that estimates the robot's lateral offset d and heading error ϕ relative to the lane center. A lane controller then computes velocity and angular rate commands to minimize these errors and keep the robot within its lane.

In this project, we extend the standard Duckietown lane following pipeline with a reinforcement learning-based obstacle avoidance module, governed by a custom Finite State Machine that determines when the robot should follow the lane, avoid an obstacle, or execute an emergency stop.

2 System Architecture: RL Concept and the FSM

2.1 Finite State Machine Overview

The high-level behavior of the robot is governed by a Finite State Machine (FSM) implemented using the `fsm_node` from the Duckietown `dt-core` stack. The FSM transitions between states based on events published by sensor nodes, and controls which ROS nodes are active at any given time.

The deployment FSM consists of three states:

- **LANE_FOLLOWING** — The robot follows the lane using the standard Duckietown controller pipeline. This is the default state. The robot drives straight through intersections without stopping.
- **OBJECT_AVOIDANCE** — Triggered when the ToF sensor detects an obstacle within the threshold distance. The TD3 RL agent takes over and sends waypoints to an avoider controller node. On successful avoidance, the FSM returns to **LANE_FOLLOWING**. On collision detection, the FSM transitions to **EMERGENCY_STOP**.
- **EMERGENCY_STOP** — Triggered when an imminent collision is detected (ToF distance ≤ 5 cm). The robot halts until the obstacle is cleared, then returns to **LANE_FOLLOWING**.

The state transition diagram is shown below:



Figure 2 Finite State Machine for training with EPISODE_RESET state.

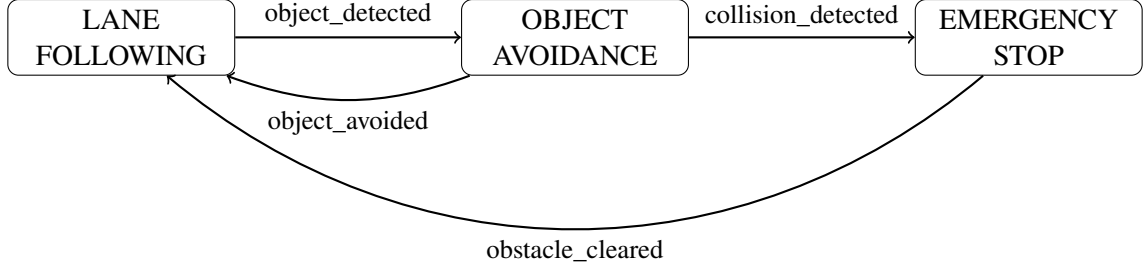


Figure 1 Finite State Machine for deployment.

We implemented a separate FSM to train the agent. In this FSM, a special state called EPISODE_RESET is introduced.

This state is triggered from the OBJECT_AVOIDANCE state when training encounters a collision with the object, times out, or successfully avoids the object. In this case, the robot stops training for that specific episode and resumes training only after the FSM is manually reset to LANE_FOLLOWING.

2.2 RL Concept

The RL agent is responsible for generating avoidance maneuvers when an obstacle is detected. The agent observes the environment through a state vector consisting of two components:

$$s = [\text{tof_distance}, \text{lane_offset}] \quad (1)$$

and outputs a continuous action:

$$a = [\omega] \quad (2)$$

where ω is the desired angular velocity. The action is converted into three waypoints and published to the avoider controller node, which physically executes the maneuver using dead reckoning.

The reward function is designed to encourage the agent to maintain a safe distance from the obstacle while successfully clearing it:

- $r = -10$ if ToF distance ≤ 0.05 m (collision)
- $r = -1$ if ToF distance < 0.20 m (critical proximity)
- $r = +10$ if ToF distance > 0.20 m while obstacle is detected
- $r = +10$ upon successful obstacle avoidance
- $r = -5$ if the robot times out

The training logic described above can be found in our RL training node in the `safe_rl` package. It is important to note, that the critical and collision distance can be modified based on preference of the developer.

3 How TD3 Works

In order to control the output of a continuous variable (angular velocity), we decided to choose a reinforcement learning algorithm that handles continuous action spaces. Twin Delayed Deep Deterministic Policy Gradient (TD3) is a model-free, off-policy reinforcement learning algorithm designed for continuous action spaces. It is an improvement over the Deep Deterministic Policy Gradient (DDPG) algorithm and addresses key issues of overestimation bias and policy instability.

3.1 Key Components

TD3 maintains the following neural networks:

- **Actor** $\pi_\phi(s)$: maps states to deterministic actions.
- **Twin Critics** $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$: independently estimate the action-value function. The minimum of the two Q-values is used during updates to reduce overestimation.
- **Target Networks**: delayed copies of the actor and critics, updated via soft updates to stabilize training.

3.2 Training Procedure

At each training timestep:

1. The agent observes the current state s and selects an action $a = \pi_\phi(s) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma)$ is Gaussian exploration noise.
2. The action is executed and a transition (s, a, r, s') is stored in a replay buffer.
3. A mini-batch is sampled from the replay buffer. The target Q-value is computed using the target networks with added clipped noise for smoothing.
4. The critics are updated by minimizing the Bellman error. The actor is updated less frequently (every 2 critic updates) by maximizing $Q_{\theta_1}(s, \pi_\phi(s))$.
5. Target networks are updated via soft updates: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$.

Our TD3 implementation was adapted from an open-source repository [8].

3.3 Deployment

In our deployment node, the trained actor weights are loaded and used for inference only. We do not include replay buffers, exploration noise, or training updates occur. The agent simply observes the state and selects the best known action.

4 Packages Added and Changed

4.1 New Packages

4.1.1 safe_rl

The `safe_rl` node is where most of our reinforcement learning logic and training lives. This package contains two nodes:

- **safe_rl_training_node**: Runs the full TD3 training loop on the physical robot. Manages episodes, replay buffer, exploration noise, reward computation, and periodic weight saving to `/data/safe_rl_weights/`.

- **safe_rl_node**: Deployment-only node. Loads pre-trained weights and runs inference. Also detects collisions and publishes `collision_detected` to trigger the FSM emergency stop.

Both nodes subscribe to the ToF sensor, lane pose, and avoider done topics, and publish waypoints to the avoider controller. Additionally, the package includes the helper class `TwinDelayed` which contains TD3, the underlying reinforcement learning algorithm that trains our robot.

4.1.2 our_fsm

Contains custom FSM configuration YAML files for both deployment (`our_fsm.yaml`) and training (`fsm_training.yaml`). The deployment FSM wires the three-state machine described in Section 2. The training FSM additionally includes an `EPISODE_RESET` state with an empty active node list, which halts all activity between training episodes.

4.2 Modified Packages

4.2.1 obstacle_detection

The `tof_obstacle_detection_node` was originally only available in the ente software stack. We ported it to daffy by removing the `fsm_controlled=True` parameter from the DTROS initializer, which is not supported in daffy. We also added input validation to filter out invalid ToF readings (values below `min_range` or at `max_range`) that would otherwise cause spurious obstacle detections:

```
def cb_tof_range(self, msg: Range):
    if msg.range >= msg.max_range or msg.range < msg.min_range:
        return # ignore invalid readings
    ...
```

4.2.2 avoider_controller_node

Similar to `obstacle_detection`, the `avoider_controller_node` originates from the ente stack and was adapted to run on daffy. This node is responsible for executing the waypoints generated by the RL agent and is therefore critical to the overall system performance.

During integration with the `safe_rl_training_node`, we identified a major issue in the waypoint checking logic used to determine when a waypoint has been reached. This logic directly affects the publication of the `avoidance_done` signal.

The controller was advancing through waypoints prematurely, particularly for the first two waypoints. This occurred because the original implementation effectively relied on the x-component of the position error, ignoring deviations in the y-direction. As a result, the robot did not properly follow the intended 2D trajectory and often reached the final waypoint from an incorrect position. Additionally, the threshold used for the final waypoint check was too strict (0.05 m), making it difficult to satisfy in practice.

Consequently, the `avoidance_done` signal was not published reliably, which negatively impacted both training and deployment.

To address this, we modified the waypoint checking logic as follows:

- For intermediate waypoints, we use the Euclidean distance $\|p_c - p_t\| < 0.05$ to ensure proper convergence in 2D space.
- For the final waypoint, we relax the condition and check $|\Delta y| < 0.15$ to account for small lateral deviations.

The updated implementation is shown below:

Listing 1 Modified waypoint checking logic in avoider controller

```
def check_point(self, current_point, target_point):
    threshold = 0.05
```

```

threshold_x = 0.15 #Fixed, loosened threshold_x

if self.iter_ == (self.len_states - 1):
    dist_x = current_point - target_point

    if abs(dist_x[1]) < threshold_x:
        return True

    return False

else:
    dist = np.sqrt(np.sum((current_point - target_point)**2 ))
    dist_x = current_point - target_point

    if (dist) < threshold:
        return True

    return False

```

This modification ensures that the robot correctly follows the full avoidance trajectory and reliably triggers the `avoidance_done` signal. This fix was essential for enabling meaningful reward signals during training, as episode completion could now be correctly detected.

4.2.3 Master Launch File

We disabled `anti_instagram` by default due to the issues with it not publishing threshold values to the line detector node. This change was motivated by observing that anti-instagram introduced a multi-second delay in image correction that caused the lane filter to operate on stale frames. Furthermore, the line detector node is remapped directly to `camera_node/image/compressed`. This is due to the fact that duckietown have legacy code has different topic names for the camera.

5 Future Implementations

Several directions remain open for future work:

- **Intersection Navigation:** The original project plan included an autonomous intersection planner (`intersection_planner`) that would detect available turns from camera segments and command the `unicorn_intersection_node` to execute SE2 trajectories via dead reckoning. This was deprioritized due to time constraints and hardware debugging challenges, but the groundwork, including the two packages, has been laid in the `ente` branch of the repository.
- **Zonotope-Based Safety Shields:** A formal safety layer using zonotope reachability analysis could be integrated to provide provable collision avoidance guarantees. The `zonopy` library provides Python primitives for zonotope arithmetic. The key challenge is constructing a 2D obstacle zonotope from the 1D ToF sensor reading, which would require additional geometric assumptions about obstacle shape.
- **Improved RL State Space:** The current state vector uses only two components. Adding the linear velocity, previous action, and a short history of ToF readings could significantly improve the agent's ability to anticipate and react to obstacles earlier.
- **Stop Line Handling:** Re-enabling the stop line bypass node or a more sophisticated intersection behavior would allow the robot to complete a full lap while respecting traffic rules. The bypass package is also available, but due to time constraints it was omitted out of the project.

- **Weight Resumption:** Currently, if training crashes midway, weights will be saved, but training can only resume from the first episode again. A future implementation is to load saved weights at training startup to enable multi-session training across bot restarts.

6 Citations

References

- [1] L. Paull et al., “Duckietown: An Open, Inexpensive and Flexible Platform for Autonomy Education and Research,” *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.
- [2] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing Function Approximation Error in Actor-Critic Methods,” *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.09477>
- [3] T. P. Lillicrap et al., “Continuous Control with Deep Reinforcement Learning,” *International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1509.02971>
- [4] M. Quigley et al., “ROS: An Open-Source Robot Operating System,” *ICRA Workshop on Open Source Software*, 2009.
- [5] ROAHM Lab, “zonopy: Zonotope Arithmetic Library for Python,” [Online]. Available: <https://github.com/roahmlab/zonopy>
- [6] Duckietown Foundation, “dt-core: Core Duckietown ROS Packages,” [Online]. Available: <https://github.com/duckietown/dt-core>
- [7] Anthropic, “Claude (Version claude-sonnet-4-6),” Large language model, 2025. [Online]. Available: <https://www.anthropic.com> Accessed: Mar. 2026.
- [8] Rafael1s, “Deep Reinforcement Learning Algorithms: TD3 Implementation,” GitHub repository, [Online]. Available: [https://github.com/Rafael1s/Deep-Reinforcement-Learning-Algorithms/tree/master/BipedalWalker-TwinDelayed-DDPG%20\(TD3\)](https://github.com/Rafael1s/Deep-Reinforcement-Learning-Algorithms/tree/master/BipedalWalker-TwinDelayed-DDPG%20(TD3)) Accessed: Mar. 2026.